The artifact I selected is my Weight Tracker Android application, which I originally created for CS-360 in Spring 2026. It was developed using Kotlin and a SQLite database. The app allows users to create an account, log in, record their weight by date, edit or delete entries, and set a goal weight. It also includes a text message notification when a user reaches their goal. The original application was built around four main files: MainActivity for login and account creation, SmsActivity for the SMS permission prompt, DatabaseHelper for database operations, and DataActivity for the main application screen.

I chose the Weight Tracker because it includes several important parts of application development, including a user interface, database storage, authentication, and device permissions. This made it a good artifact for demonstrating improvements in both software design and security. My original code review also identified several problems that I wanted to address. I wanted my portfolio to show not only that I could create an application, but also that I could go back to existing code, identify weaknesses, and improve it.

I made several enhancements based on the issues I identified during my code review.

Security: The original application stored passwords as plain text. I replaced this with salted PBKDF2-HMAC-SHA256 password hashing using 600,000 iterations and a randomly generated 16-byte salt for each user. Password verification also uses a constant-time comparison. The stored password information includes the algorithm, iteration count, salt, and hash, making it possible to adjust the work factor in the future. I chose PBKDF2 because it is supported by Android's cryptography libraries without requiring an additional dependency. I also stopped trimming passwords, made login display the same error message for an unknown username and an incorrect password, disabled application backups, and restricted access to internal screens.

Data isolation: The original weights table did not associate entries with individual users. This meant that multiple users could see each other's weight records and shared the same goal. I added a user_id column to associate weight entries with their owners, moved goal weights into a separate per-user table, and updated database queries so that users can only view, edit, or delete their own information.

Database migration: The original database upgrade process deleted existing tables, which meant that users could lose their information when the database was updated. I replaced that behavior with a version 1 to version 2 migration that hashes existing passwords and preserves weight entries and saved goals.

Application structure: The original DataActivity handled several responsibilities in one large class. I separated those responsibilities into a screen, ViewModel, services, and a repository interface with a SQLite implementation. I also replaced the manually created table rows with a RecyclerView. The text message functionality now uses an interface, which allows the goal logic to be tested without actually sending a text message.

Input validation and correctness: I added validation for weight entries and dates. Weights must be numeric and within a valid range, while dates must be valid ISO-formatted dates that are not in the future. The input dialogs also remain open when an error occurs and display a message explaining what needs to be corrected. I improved the goal-checking logic so that it recognizes passing a goal instead of requiring an exact match, and prevents repeated notifications for the same goal.

Responsiveness: I moved database operations and password hashing off the main thread using Kotlin coroutines. This helps prevent longer operations from interfering with the application's responsiveness.

Testing: I replaced the empty template tests with 32 unit tests using an in-memory fake repository. These tests cover password hashing, input validation, goal logic, login, adding and editing entries, deleting entries, and ensuring users cannot access or modify another user's records.

The parts of the enhanced application that best demonstrate my skills are the password hashing implementation, database migration, user-specific repository queries, and service classes with their associated tests. Together, these improvements show how software design decisions can affect security, reliability, and maintainability.

This enhancement contributes to several of the Computer Science program outcomes. My strongest areas of progress are software development practices, communication, and security.

Outcome 1 – Collaboration: I improved the organization and documentation of the application so that another developer can more easily understand its structure and design decisions. Although I did not complete a collaborative development project as part of this enhancement, making the code easier to understand and maintain supports future collaboration.

Outcome 2 – Communication: I documented the changes made to the application and explained the reasoning behind important design decisions. This narrative also communicates the improvements and what I learned from the process.

Outcome 3 – Algorithms and Data Structures: The goal-checking logic demonstrates the use of conditional logic to determine whether a user has reached or passed their goal. I also considered the trade-off between password security and login performance when choosing the PBKDF2 iteration count. I plan to demonstrate more of this outcome through my separate Algorithms and Data Structures enhancement.

Outcome 4 – Software Development Practices: I used Kotlin coroutines, a ViewModel, RecyclerView with DiffUtil, parameterized SQLite queries, and a versioned database migration. These changes improved the organization, performance, and maintainability of the application.

Outcome 5 – Security: I identified several security weaknesses in the original application, including plain-text passwords, shared user data, and potential exposure through application

backups. I addressed these issues through password hashing, user-specific database queries, and changes to the application's backup and screen access settings.

Overall, my outcome coverage is still on track with my original ePortfolio plan. This enhancement gave me more opportunities to demonstrate software engineering and security practices, while I plan to address additional algorithm and data structure concepts in the next enhancement.

The biggest thing I learned from this enhancement is that fixing one problem can affect several other parts of an application. Replacing plain-text passwords with password hashing required changes to the database structure. Those changes also required a migration, which meant figuring out how to handle information created under the original database design. Since the original weight entries were not associated with individual users, I had to decide how to assign that existing information during the migration. I assigned the older entries to the earliest account. When I tested the upgrade, my own test account did not show its entries because the existing data had been assigned to a different account I had created earlier. I used the Database Inspector to check which user owned the records and correct the issue. This helped me understand that database migrations involve more than changing a schema. They also require considering how existing user information will be affected.

I also learned how much separating application responsibilities can help with testing and maintenance. By moving database operations and application logic into separate classes, I could test things like user separation and goal calculations without needing to run the entire application in an emulator. This made it easier to identify problems before connecting everything to the screens.

Some of the challenges I encountered were smaller development issues that still taught me important lessons. At one point, I received an unresolved reference error because I had accidentally placed a new service file in the test folder instead of the main source folder. This reminded me that file organization matters because it determines which parts of the project can access a class. I also noticed that the rows in my new list were displaying with a blue background because I had reused a drawable intended for the header. I created a separate border style to correct that. Another challenge was keeping the old and new database code from interfering with each other while I replaced the original screens. I had to make changes in an order that allowed the project to continue building as I worked. I also ran into issues using the Database Inspector because it rejected queries with trailing semicolons.

If I continued working on this application, I would consider adding Argon2id password hashing, creating an instrumented test for the database migration, and adding a logout option. Overall, this enhancement helped me understand that improving an application involves more than adding features. It also means thinking about how the different parts work together, how user information is protected, and how the application can be maintained in the future.